

CS2422 Assembly Language & System Programming

September 25, 2003

Today's Topics

- Indirect Addressing
 - Now you can handle array!

Study Guide

- Section 4.4: Indirect Addressing

Direct-Offset Addressing

- During last lecture, we discussed Direct-Offset operands:
- Problem: the offset is fixed.
 - Can't handle array index, like $A[i]$

Indirect Addressing

- The solution? The memory address must be a variable too! → Store it in a register!
- Compare these:
 - `MOV AL, [10000h]`
 - `MOV AL, [Var1+1]`
 - `MOV AL, [ESI]`

OFFSET Operator

- But...How do we get the address?
 - For example: “`MOV ESI, Var1`” moves the value of `var1`, not its address.
- Answer: Use the `OFFSET` operator to obtain the address.
 - `MOV ESI, OFFSET Var1`

Array – An Example

```
.data
arrayB BYTE 10h, 20h, 30h
.code
    mov ESI, OFFSET arrayB
    mov AL, [ESI] ; first byte
    INC ESI
    add AL, [ESI] ; second byte
    INC ESI
    add AL, [ESI] ; third byte
```

Array Index

- So, can you modify the code in last slide to implement array index like arrayB[i]?
(Assume i is stored in a register, e.g., ECX.)

Wait! There is an easier way...

Indexed Operands

A Few Examples:

- `[arrayB+ESI]` or simply `arrayB[ESI]`
- `[ESI+2]`, `[ESI+4]`, ...etc.

Pointers

- Now we know that we can store an address in a register.
- Can we store it in a variable (in memory) too?